R3st0

Itération 2
Fiabiliser le site et choisir les priorités

Séances du 5 et 12 octobre 2026
8 heures de travail en classe

Vous poursuivez la reprise de R3st0.fr, le site de critiques de restaurants installé pendant l'itération 1. Plusieurs parcours doivent être corrigés et de nouvelles améliorations sont demandées. Votre équipe doit livrer une version plus fiable, en tenant compte du temps disponible.

Votre mission

Choisir avec le Product Owner les besoins les plus utiles, diagnostiquer les problèmes retenus, réaliser les modifications et démontrer leur fonctionnement. L'enseignant joue le rôle de Product Owner et arbitre la valeur métier et les priorités ; l'équipe estime le travail et organise sa réalisation.

Le backlog proposé est volontairement plus important que votre capacité de réalisation. Vous ne devez pas tout terminer. Vous devez négocier un objectif réaliste et rendre visibles vos résultats ainsi que les besoins reportés.

À conserver	À faire évoluer
Le dépôt partagé, main et dev, les comptes GitLab et les règles de relecture.	Les Issues, le tableau de suivi et le périmètre choisi avec l'enseignant.
L'application PHP MVC et la base resto2.	Les parties du code et, si nécessaire, de la base concernées par vos tickets.
Le pied de page réalisé en itération 1 et les parcours déjà fonctionnels.	La fiabilité et la qualité des parcours retenus pour cette itération.



Un fonctionnement professionnel à petite échelle

À deux ou trois, certaines coordinations paraîtront simples. Utilisez néanmoins les pratiques proposées comme si votre travail devait être repris par une équipe plus grande : décisions visibles, tickets compréhensibles, tests reproductibles et code relu. Les rôles et points d'équipe constituent ici une adaptation pédagogique de Scrum.

Le résultat attendu le 12 octobre

Une version démontrable, un périmètre accepté ou explicitement reporté avec le Product Owner, et des preuves de travail dans GitLab. Le nombre de tickets fermés ne suffit pas à juger la réussite : la valeur des choix, la qualité et la coopération comptent.

> ⚠ L'itération 3 du 2 novembre portera sur les types de cuisine. N'anticipez pas cette évolution pendant cette itération sans arbitrage de l'enseignant.

Choisir et organiser le travail

Commencer par consulter le Product Owner

1. Parcourez les douze besoins des pages suivantes et les anomalies déjà relevées en itération 1.
2. Faites une première investigation et estimez la complexité : S (small), M (medium) ou L (large). Repérez les dépendances, les incertitudes et le temps nécessaire aux tests et à la relecture. Une estimation est une aide à la décision, pas une promesse de durée exacte.
3. Présentez votre proposition à l'enseignant avant de vous engager. Demandez quels parcours comptent le plus, quels défauts sont les plus gênants et quelles règles restent à préciser. Faites confirmer ce qui peut être reporté. Justifiez de manière convaincante vos décisions.
4. Formulez un objectif commun en une phrase. Choisissez un périmètre compatible avec les huit heures, y compris diagnostic, échanges, intégration et démonstration. Si tout paraît prioritaire, demandez un ordre explicite et proposez un compromis.

Exemple de formulation : « À la fin de l'itération, un visiteur pourra … ; nous vérifierons ce résultat avec … ». Le contenu doit venir de votre échange avec le Product Owner.

Rendre les décisions visibles dans GitLab

Créez le Milestone « Itération 2 - Fiabilisation », échéance le 5 octobre 2026. Rattachez-y les Issues retenues. Les autres restent dans le backlog. Les repères I2-01 à I2-12 du sujet ne sont pas les numéros attribués par GitLab.

Chaque Issue retenue contient : besoin utilisateur, critères d'acceptation, priorité validée, complexité, responsable, diagnostic puis tests. Utilisez À faire, En cours, À relire/tester et Terminé. Limitez à un ticket en réalisation par personne ; aidez à tester ou débloquer avant d'en commencer un autre.

Faire vivre les rôles et les points courts

Désignez un facilitateur, dans le rôle de Scrum Master. Il facilite les échanges, rend les obstacles visibles et veille aux relectures ; il ne devient pas le chef de l'équipe. Tous les membres développent, testent et expliquent leurs contributions.

Tenez un point de cinq minutes au démarrage ET au milieu de chaque séance : où en sommes-nous par rapport à l'objectif, quel est le prochain travail utile, quel obstacle nécessite de l'aide ? Actualisez le tableau et terminez par une décision concrète. Ces points adaptés au rythme scolaire reprennent l'intention du Daily Scrum.

> 🗎 Gardez une trace de ces échanges dans un fichier .md du dossier docs.

N’attendez pas le point suivant si vous êtes bloqués plus de quinze minutes. Consultez un équipier, puis l’enseignant si nécessaire. Une règle métier ambiguë se clarifie avec le Product Owner ; une difficulté technique se travaille d’abord en équipe.

Adapter le périmètre sans le perdre de vue

Si une estimation change ou si une demande arrive, proposez ce qui sera simplifié ou reporté en échange. Faites valider l’arbitrage par le Product Owner et consignez la date, la raison et l’effet sur l’objectif dans docs/iteration2.md. Un besoin reporté reste ouvert et ne compte pas comme terminé.

Le backlog proposé

L’ordre ci-dessous sert à repérer les besoins. Il ne fixe pas leur priorité. Le Product Owner vous aide à les classer, puis votre équipe choisit avec lui un périmètre réalisable.

Repère	Besoin	Valeur attendue
I2-01	Signaler une action inconnue	Comprendre pourquoi la page demandée est introuvable.
I2-02	Moderniser le stockage des mots de passe	Mieux protéger les comptes sans perdre les données liées.
I2-03	Corriger le classement de l’accueil	Comparer les restaurants à partir de leurs notes moyennes.
I2-04	Afficher la note moyenne sur une fiche	Lire une valeur numérique en complément des icônes.
I2-05	Valider le code postal	Éviter les critères de recherche incorrects.
I2-06	Gérer une adresse déjà inscrite	Comprendre pourquoi une inscription est refusée.
I2-07	Sécuriser les requêtes de connexion	Traiter les saisies comme des données SQL.
I2-08	Réparer la recherche multicritère	Trouver des restaurants avec plusieurs critères.
I2-09	Expliquer une recherche sans résultat	Pouvoir corriger facilement sa recherche.
I2-10	Prévoir une image de remplacement	Garder une fiche lisible lorsqu’une photo manque.
I2-11	Fiabiliser la continuité de connexion	Naviguer avec un état de connexion cohérent.
I2-12	Améliorer la consultation sur mobile	Consulter liste et fiche sur un écran étroit.

Avant de corriger

Un signalement est un point de départ à vérifier sur votre version. Reproduisez le comportement, identifiez sa cause et localisez les fichiers, classes ou méthodes concernés. Si le défaut est déjà corrigé, apportez la preuve au Product Owner : ne créez pas une modification artificielle.

Des règles à préciser ensemble

Les critères des pages suivantes forment une base d’acceptation. Les questions marquées « À clarifier » doivent recevoir une réponse avant le développement du besoin concerné. Consignez la décision dans l’Issue, afin que le relecteur teste la même règle que le développeur.

Besoins de maintenance 1 à 8

I2-01 : Signaler une action inconnue

Besoin : lorsqu'un visiteur demande une action inexistante, le site doit expliquer la situation avec un message clair et lui permettre de poursuivre sa navigation avec un lien.

Acceptation : index.php?action=blob affiche un message compréhensible ; l'utilisateur peut rejoindre l'accueil ; aucune erreur technique ni chemin de fichier interne n'est affiché. Une action valide et l'accès sans paramètre continuent à fonctionner.

À clarifier : le message est-il présenté dans l'accueil ou dans une page dédiée ? Le comportement retenu doit être cohérent avec la navigation existante.

I2-02 : Moderniser le stockage des mots de passe

Besoin : remplacer le stockage historique reposant sur crypt et un sel fixe par un hachage adapté aux mots de passe, avec password_hash et PASSWORD_DEFAULT, puis une vérification avec password_verify. Il s'agit de hachage, pas de chiffrement réversible.

Acceptation : un compte créé utilise le nouveau mécanisme ; le bon mot de passe permet la connexion et un mauvais la refuse. La colonne SQL accepte un hachage de 255 caractères. Aucun mot de passe en clair n'est enregistré dans la base, les journaux ou Git.

À clarifier : comment traiter les comptes existants ? Proposez une migration après vérification de l'ancien mot de passe, ou une réinitialisation encadrée des comptes de test. Un ancien hachage ne permet pas de retrouver le mot de passe. Les identifiants utilisateurs, avis et favoris doivent être conservés. Testez aussi un compte existant et documentez la transition.

I2-03 : Corriger le classement de l'accueil

Besoin : les quatre restaurants mis en avant doivent être classés selon leur note moyenne, et non selon la somme des notes reçues.

Acceptation : le résultat correspond à un calcul de contrôle dans la base ; les notes NULL sont ignorées ; les restaurants sans note sont exclus ; au maximum quatre restaurants sont affichés. Les données ne sont ni modifiées ni codées en dur pour obtenir le classement attendu.

À clarifier : quelle règle départage les moyennes égales ? Appliquez une règle stable et testez des restaurants ayant des nombres de notes différents, ainsi qu'une égalité.

I2-04 : Afficher la moyenne sur une fiche

Besoin : afficher une note moyenne numérique près des icônes de notation, afin de mieux distinguer les restaurants.

Acceptation : le calcul conserve la partie décimale et utilise seulement les notes renseignées. La valeur affichée correspond à la moyenne SQL. En l'absence de note, un message explicite remplace une fausse moyenne de zéro. Les icônes restent cohérentes avec la règle d'affichage.

À clarifier : combien de décimales faut-il afficher ? Distinguez la valeur calculée de son arrondi de présentation. Testez une moyenne non entière et un restaurant sans note.

I2-05 : Valider le code postal

Besoin : dans la recherche par adresse et la recherche multicritère, un code postal renseigné doit contenir exactement cinq chiffres. Un champ vide reste permis pour une recherche sans ce filtre.

Acceptation : une validation côté serveur refuse une valeur invalide avec un message lisible. Le code postal reste une chaîne, afin de conserver un zéro initial. Une aide ou une validation dans le navigateur peut compléter la validation serveur.

Tests minimaux : vide, 75001, 01000, quatre chiffres, six chiffres et présence d'une lettre. Vérifiez le refus côté serveur même lorsque la validation du navigateur est contournée. Faites préciser le traitement des espaces autour de la saisie.

I2-06 : Gérer une adresse déjà inscrite

Besoin : un visiteur qui tente de créer un compte avec une adresse électronique déjà utilisée doit comprendre pourquoi son inscription ne peut pas aboutir.

Acceptation : message compréhensible, aucune erreur SQL brute, aucun second compte créé et aucune modification du compte existant. Une adresse nouvelle et valide permet toujours l'inscription. La contrainte d'unicité dans la base est conservée et une éventuelle violation est gérée proprement.

À clarifier : formulation du message et possibilité de retourner vers la connexion. Testez une inscription nouvelle et une inscription avec une adresse déjà présente.

I2-07 : Sécuriser les requêtes de connexion

Besoin : les saisies de connexion doivent être traitées comme des valeurs, sans pouvoir modifier la structure des requêtes SQL exécutées.

Acceptation : les requêtes du parcours de connexion utilisant les saisies sont préparées et leurs valeurs sont liées avec PDO. Des caractères spéciaux ne provoquent ni erreur SQL visible ni connexion non autorisée. Le parcours normal fonctionne toujours et un échec de connexion affiche un message générique.

Vérification : sur votre application locale et une base de test sauvegardée, essayez une saisie contenant une apostrophe, puis vérifiez le refus et l'intégrité des données. Complétez ce test par une relecture du code SQL : un seul essai ne prouve pas la sécurité du parcours. N'utilisez pas de requête destructrice. Coordonnez ce besoin avec I2-02 s'ils sont tous deux retenus.

I2-08 : Réparer la recherche multicritère

Besoin : rechercher des restaurants en combinant les champs existants de nom et d'adresse, sans obtenir systématiquement une liste vide.

Acceptation : chaque critère non vide est pris en compte ; les critères se combinent avec ET ; un champ vide n'ajoute pas de filtre. La recherche fonctionne sans type de cuisine et sans dépendre d'une table réservée à une évolution future.

À clarifier : correspondance partielle ou exacte pour le nom, la voie et la ville ; comportement lorsque tous les champs sont vides. Testez un seul critère, deux critères compatibles, deux incompatibles et tous les champs vides. Coordonnez les règles avec I2-05 et I2-09.

Améliorations à arbitrer 9 à 12

I2-09 : Expliquer une recherche sans résultat

Besoin : lorsqu'une recherche ne trouve aucun restaurant, le visiteur doit comprendre le résultat et pouvoir ajuster sa demande facilement.

Acceptation : un message distingue l'absence de résultat d'une erreur technique. Les critères saisis restent visibles dans le formulaire ou sont rappelés clairement. Le visiteur peut relancer une recherche et réinitialiser les critères sans modifier l'URL à la main.

À clarifier : quels formulaires sont couverts par ce ticket ? Testez un résultat vide, puis une recherche réussie et une réinitialisation. Si I2-08 est retenu, partagez les règles et évitez deux modifications concurrentes du même formulaire.

I2-10 : Prévoir une image de remplacement

Besoin : une photo manquante ne doit pas produire une icône cassée ou rendre la fiche difficile à lire.

Acceptation : une image de remplacement ou un bloc explicite apparaît lorsqu'une photo est absente. Une photo disponible reste affichée. Le remplacement conserve une taille cohérente et possède un texte alternatif adapté.

À clarifier : traiter seulement la photo principale ou aussi les autres photos ? Reproduisez le cas sur un jeu de test, puis restaurez l'état initial. Ne supprimez pas un restaurant ni ses avis pour contourner une photo manquante. Documentez toute nouvelle image et son origine.

I2-11 : Fiabiliser la continuité de connexion

Besoin : après s'être connecté, un utilisateur doit conserver un état de connexion cohérent lorsqu'il navigue entre l'accueil, la liste et une fiche, puis pouvoir se déconnecter.

Acceptation : vérifiez la connexion, plusieurs changements de page, une actualisation puis la déconnexion. Après déconnexion, les actions réservées à un utilisateur connecté ne sont plus accessibles. Un nouveau visiteur peut consulter les pages publiques sans erreur. Commencez par reproduire le parcours sur votre version. Localisez toute anomalie entre session, contrôleurs et accès aux données. Si le parcours fonctionne déjà, proposez au Product Owner de valider le besoin par les tests, sans modifier inutilement le code. La fonctionnalité « se souvenir de moi » ne fait pas partie de ce ticket.

I2-12 : Améliorer la consultation sur mobile

Besoin : pouvoir consulter la liste des restaurants et une fiche sur un écran étroit, sans perdre une information ou un accès utile.

Acceptation : à une largeur de 360 pixels, les deux pages ne présentent pas de défilement horizontal global ; textes et images restent lisibles ; les liens essentiels sont accessibles. Vérifiez aussi que l'affichage de bureau reste utilisable.

À clarifier : quels éléments de navigation doivent être accessibles et quelles pages exactes servent à l'acceptation ? Utilisez le mode adaptatif du navigateur et consignez les dimensions testées. Une refonte graphique complète n'est pas attendue.

Développer relire et intégrer

Une trace courte pour chaque ticket retenu

Dans l'Issue ou dans docs/iteration2.md avec un lien vers l'Issue, consignez : scénario de reproduction et résultat attendu ; cause et localisation ; solution choisie et justification ; tests avec données, résultats observés et conclusion. Une capture complète une explication, elle ne la remplace pas.

Pour une évolution de base, fournissez un script SQL versionné et les instructions d'exécution dans le README. Testez la mise à jour sur une copie ; préservez les données existantes. Chaque équipier doit pouvoir reproduire le nouvel état sans réimporter aveuglément toute la base.

Le cycle Git pour chaque réalisation

1. Mettez votre branche dev locale à jour, puis créez une branche dédiée, par exemple issue-23-recherche-multiple. Utilisez le vrai numéro GitLab de l'Issue.
2. Réalisez de petits commits explicites. Poussez la branche et ouvrez une Merge Request vers dev ; indiquez le besoin, les changements, les tests et Refs #23. Une MR en brouillon peut rendre le travail visible avant sa fin.
3. Avant la relecture finale, récupérez dev et fusionnez sa version à jour dans votre branche de ticket. Résolvez les conflits, relancez les tests concernés puis poussez la branche actualisée.
4. Un autre membre lit la différence de code et teste la branche localement avec la procédure fournie. Il laisse un retour précis. Corrigez les points relevés avant l'intégration dans dev.
5. Intégrez progressivement les MR validées. Vérifiez aussi le résultat sur dev. Fermez manuellement l'Issue lorsque les critères sont vérifiés et l'acceptation obtenue ; une fusion ne suffit pas à prouver que le besoin est satisfait.

> ⚠ Ne créez pas une branche commune « iteration2 » qui remplace dev. Ne poussez pas directement les réalisations sur main. Les réglages privés d'IDE, clés et secrets personnels restent hors du dépôt.

Une définition commune d'un ticket terminé

Contrôle	Preuve attendue
Besoin satisfait	Critères vérifiés et règle métier confirmée par le Product Owner.
Solution comprise	Cause identifiée, modification expliquée et relecture par un équipier.
Fonctionnement vérifié	Cas normal, cas limite ou invalide, puis parcours voisins testés.
Travail intégré	MR validée vers dev et résultat vérifié après fusion.
Reprise possible	Issue à jour ; documentation et script SQL fournis si nécessaires.

Vérifier aussi ce qui fonctionnait déjà

Après intégration, contrôlez au minimum accueil, liste, fiche et pied de page. Ajoutez connexion, inscription ou recherche selon les zones modifiées. Distinguez une anomalie déjà connue d'une régression introduite. Aucun changement ne doit être présenté comme validé si ses tests restent à faire.

Ressources PHP : php.net/manual/fr/function.password-hash.php, php.net/manual/fr/function.password-verify.php, php.net/manual/fr/pdo.prepare.php

Proposition d'organisation et remise finale

Lundi 5 octobre de 14h à 18h

Horaire indicatif	Étape
14h00–14h15	Point d'équipe, vérification de dev et lecture des besoins.
14h15–14h45	Investigation, estimations et échange avec le Product Owner ; objectif et périmètre retenus.
14h45–16h00	Réalisation des premiers tickets et ouverture des MR.
16h00–16h15	Pause et point d'équipe de cinq minutes ; ajustement si nécessaire.
16h15–17h35	Développement, tests croisés et intégrations progressives.
17h35–18h00	Démonstration intermédiaire, retour du Product Owner et préparation de la suite.

À 18h : tableau à jour, travail poussé, déployé pour le Product Owner et état des tests visible. Notez ce qui fonctionne sur dev, les blocages et la prochaine action. Le travail incomplet peut rester dans une MR en brouillon ; une release finale n'est pas demandée à cette étape.

Lundi 12 octobre de 14h à 18h

Horaire indicatif	Étape
14h00–14h15	Point d'équipe et confirmation des priorités avec le Product Owner.
14h15–16h00	Réalisation et relecture des tickets retenus.
16h00–16h15	Pause et point d'équipe, arbitrage des reports avec le Product Owner.
16h15–17h15	Achèvement, tests et intégration.
17h15–17h40	Arrêt des nouveaux développements, contrôles de la version et préparation de la livraison.
17h40–18h00	Déploiement pour démonstration finale

Ce que votre équipe remet

Dans GitLab : les Issues et MR à jour ; une MR de dev vers main pour la version validée ; le tag v0.2.0 sur le commit livré dans main et une Release indiquant les changements, limites et besoins reportés. Conservez dev et ne faites pas de squash de cette branche durable. Gardez le travail non validé hors de la version livrée.

Dans docs/iteration2.md : objectif et périmètre initial ; décisions datées et ajustements ; liens vers les tickets et tests ; bilan livré ou reporté ; contributions de chacun ; rétrospective avec un point réussi et une difficulté. Mettez le README à jour si la procédure de lancement ou de mise à jour change.

Pendant la démonstration

Montrez un parcours utilisateur représentatif sur la version livrée, un cas limite et les preuves de validation. Expliquez un arbitrage et un changement technique. Chaque membre doit pouvoir présenter sa contribution. L'enseignant observe la qualité du résultat, la justification des choix, la traçabilité et la coopération.